```
    try {
      const { text } = req.body;
      const { data: completion, response } =
await this.openai.chat.completions.
        .create({
          messages: [
            {
              role: "system",
              content:
                "I'm happy to assist you in any way I can.
How can I be of service today?",
            },
            { role: "user", content: text },
          ],
          model: "gpt-3.5-turbo",
          stop: null,
          max_tokens: 150,
        })
        .withResponse();
      const message = {.
        id: completion.id,
        created: completion.created,
        role: "assistant",
        content: completion.choices[0].message.content,
      };
      res.json({ message });
    } catch (e) {
      console.error(e);
      res.status(500).send("Internal server error");
    }
  }
}
```

Generates response using OpenAI; utilizes the stored OpenAI client to generate a response using the chat.completions.create function

Specifies model for response generation; sets the model property to gpt-3.5-turbo to generate the response

Removes stop sequences; sets the stop parameter to null to ensure the generated response is more conversational and avoids stopping at predefined sequences

Limits response length; sets the max_tokens parameter to limit the response length to 150 tokens (can be adjusted based on requirements)

Sends response as JSON; sends the formatted response message as JSON data back to the client

Implements error handling; if an error occurs, it's logged, and a generic Internal Server Error response is sent to the client.

Formats response message; constructs a response message object with relevant properties like ID, creation timestamp, role ("assistant"), and the actual response content extracted from the OpenAI response

Within the `handleRequest` function, we use the `chat.completions` endpoint of the OpenAI API. This specific endpoint is designed to generate conversational responses tailored to a provided chat history. By using this endpoint, we can analyze the user's input context and craft a relevant, informative response. OpenAI, of course, provides several other endpoints tailored to generating different kinds of requirements.

We are using the specific configuration values that we used for `chat.completions` API for several reasons:

- `stop`—Setting the stop parameter to `null` instructs the OpenAI API to continue generating text until it reaches its internal completion criteria. This enables more elaborate responses that may go beyond a simple one-sentence answer.
- `max_tokens`—This parameter acts as a control mechanism for the response length. Setting it to 150 limits the generated response to a maximum of 150